

TASK 10 - NumPy Indexing and Slicing

FULL AND COMPLETE ASSESSMENT

1. Task Overview

This task focuses on NumPy indexing and slicing. It demonstrates how to access individual elements, rows, columns, ranges, and filtered numerical values from NumPy arrays.

2. Objective

Develop the ability to efficiently access numerical data stored in arrays and understand how indexing and slicing are used in data processing.

3. Tools and Technologies

Python, NumPy, and Jupyter Notebook.

4. NumPy Array Basics

A NumPy array is a numerical data structure. A 1D array stores values in sequence, while a 2D array stores values in rows and columns.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60])

matrix = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])
```

5. One-Dimensional Indexing

Indexing means accessing a specific element. NumPy uses zero-based indexing, so the first element has index 0.

```
arr[0]      # 10
arr[2]      # 30
arr[-1]     # 60
arr[-2]     # 50
```

6. Positive and Negative Indexing

Positive indexes count from the beginning. Negative indexes count from the end. The last element is -1.

```
# Positive
arr[0]      # first
arr[1]      # second

# Negative
arr[-1]     # last
arr[-2]     # second last
```

7. Index Position Reference

Value	10	20	30	40	50	60
Positive	0	1	2	3	4	5
Negative	-6	-5	-4	-3	-2	-1

8. Two-Dimensional Indexing

For a 2D array, use `array[row, column]`. Both positions start at 0.

```
matrix[0, 0]    # 10
matrix[1, 2]    # 60
matrix[2, 1]    # 80
```

9. Selecting Rows

Use `array[row_index, :]` to select an entire row.

```
matrix[0, :]    # [10 20 30]
matrix[1, :]    # [40 50 60]
matrix[-1, :]   # [70 80 90]
```

10. Selecting Columns

Use `array[:, column_index]` to select an entire column.

```
matrix[:, 0]    # [10 40 70]
matrix[:, 1]    # [20 50 80]
matrix[:, 2]    # [30 60 90]
```

11. What is Slicing?

Slicing means selecting multiple elements, a range, or a pattern. The basic syntax is `array[start:stop:step]`. The stop position is normally excluded.

12. Basic Slicing Examples

```
arr[:3]         # first three elements
arr[1:4]        # indexes 1, 2, 3
arr[2:5]        # indexes 2, 3, 4
arr[::-2]       # every second element
arr[::-1]       # reverse
arr[-3:]        # last three elements
```

13. Start, Stop and Step

Start defines where selection begins, stop defines where it ends, and step controls the interval.

```
arr[1:6:2]
# start = 1
# stop = 6
# step = 2
```

14. Two-Dimensional Slicing

2D slicing can select multiple rows, multiple columns, or a rectangular block.

```
matrix[:2, :]   # first two rows
matrix[:, 1:]   # columns from index 1
matrix[1:3, 1:3] # selected block
matrix[:2, :2]  # first two rows/columns
```

15. Reverse Slicing

A negative step reads the array from right to left.

```
arr[::-1]
arr[5:1:-1]
```

16. Boolean Indexing

Boolean indexing filters values according to a condition. It is useful when working with numerical datasets.

```
values = np.array([55, 68, 72, 80, 88, 95])

values[values > 80]
# [88 95]
values[values >= 70]
```

```
# [72 80 88 95]
```

17. Practical Student Marks Dataset

Rows represent students and columns represent subjects. Indexing and slicing can extract specific student or subject data.

```
marks = np.array([
    [78, 82, 91],
    [65, 74, 80],
    [88, 90, 95],
    [55, 68, 72]
])

marks[0, :]      # first student
marks[:, 1]      # second subject
marks[:2, :]     # first two students
marks[-2:, :]    # last two students
marks[marks > 80] # marks above 80
```

18. Indexing vs Slicing

Indexing selects a specific element, while slicing selects multiple elements, a range, or a pattern.

Feature	Indexing	Slicing
Purpose	Specific element	Range or pattern
Example	arr[2]	arr[1:4]
2D example	matrix[1,2]	matrix[:,2:]
Typical use	Single value	Subset of data

Easy rule: Indexing = one specific value. Slicing = multiple values or a range.

19. Common Mistakes

1. Forgetting that indexing starts at 0.
2. Assuming the stop index is included in a slice.
3. Mixing up row and column positions.
4. Forgetting that -1 means the last element.
5. Using an invalid index outside the array range.

20. Step-by-Step Problem Solving

First identify whether the array is 1D or 2D. Next identify the required element, row, column, or range. If selecting a range, decide the start, stop, and step. Finally check the returned data.

21. Interview Questions and Answers

What is zero-based indexing? The first element of a NumPy array has index 0.

What is negative indexing? It accesses elements from the end; -1 is the last element.

What is slicing? Selecting a range or pattern using start, stop, and step.

How do you select a row? Use array[row_index, :].

How do you select a column? Use array[:, column_index].

What does arr[::-1] do? It reverses the array.

What is Boolean indexing? It filters array values using a condition.

Why is slicing useful? It efficiently selects required subsets of data.

22. Quick Revision

Indexing	-> arr[2]
Negative	-> arr[-1]
Row	-> matrix[0, :]
Column	-> matrix[:, 1]
Range	-> arr[1:4]
Every second	-> arr[::2]
Reverse	-> arr[::-1]
Last 3	-> arr[-3:]
Condition	-> arr[arr > 80]

23. Complete Mini Program

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60])

print("First:", arr[0])
print("Last:", arr[-1])
print("First three:", arr[:3])
print("Every second:", arr[::2])
print("Reverse:", arr[::-1])

marks = np.array([
    [78, 82, 91],
    [65, 74, 80],
    [88, 90, 95],
    [55, 68, 72]
])

print("First student:", marks[0, :])
print("Second subject:", marks[:, 1])
print("Marks above 80:", marks[marks > 80])
```

24. Expected Learning Outcome

After completing this task, the learner should be able to access individual NumPy values, select rows and columns, perform 1D and 2D slicing, use negative indexes, reverse arrays, and filter numerical data with Boolean conditions.

25. Top 5 Skills

1. NumPy
2. Array Indexing
3. Array Slicing
4. Numerical Data Processing
5. Python Data Analysis

26. Feedback

The task was very helpful for understanding how NumPy indexing and slicing are used to access numerical data efficiently. I learned how to select individual elements, rows, columns, and ranges using positive and negative indexing. The practical slicing examples and student marks dataset improved my understanding of array manipulation and strengthened my Python and data-analysis skills.